P.Vishwa Nandan(192473015)

then task12

TASK 12 – Re-evaluate and Adapt the Design

Marks: 8

This is the final task of your Java assignment.

In this task, the university introduces new requirements. We must analyse our existing Smart-Parking System, make necessary changes and explain why the updated design is better.

According to your assignment, the new requirements are:

1. **E-bikes:** Charged a flat parking fee.
2. **Delivery vans:** Charged at double the car rate between 9 AM and 5 PM.
3. **Visitors:** Can purchase a one-day parking pass.
4. **Parking facility:** Extended from 3 floors to 5 floors, with different slot capacities on each floor.

We must complete four sections:

- A. Impact Analysis – 2 Marks
- B. Evaluation of Original Design – 2 Marks
- C. Adaptation or Redesign – 2 Marks
- D. Evidence-Based Defence – 2 Marks

A. Impact Analysis

2 Marks

1. Changes from the Original Problem

The original parking system supported two-wheelers, cars and electric cars. It also supported monthly parking passes for students and staff.

The university introduces the following changes:

1. New Vehicle Categories

Two new vehicle categories are introduced:

- E-bikes are charged a flat parking fee regardless of parking duration.
- Delivery vans are charged double the car parking rate between 9 AM and 5 PM.

2. New Pricing Rule

The system must support different pricing methods, including hourly pricing, flat pricing and time-based pricing.

3. One-Day Parking Pass

Visitors can purchase a one-day parking pass that allows them to park free of charge on the pass's valid date.

4. Increased Parking Capacity

The parking facility is extended from 3 floors to 5 floors. Each floor has a different number of parking slots.

2. Classes, Methods and Data Structures Affected

The following components are affected:

Vehicle Class:

- Must support different pricing rules.
- New vehicle categories must be added.

Car Class:

- The car hourly rate is used as the base rate for delivery vans.

ParkingFacility Class:

- Must support five floors.
- Must allocate and release slots across different floors.

ParkingTicket Class:

- Must store the applicable parking pass information.

ParkingPass Class:

- Must support both monthly passes and one-day visitor passes.

Data Structure:

- A jagged two-dimensional array is used because different floors have different numbers of parking slots.

3. Why Different Slot Capacities Affect the Array Design

A normal rectangular two-dimensional array provides the same number of slots on every floor.

However, our updated parking facility has different slot capacities.

Therefore, a jagged array is more suitable.

Example:

Parking Facility

Floor 0 → 20 slots

Floor 1 → 25 slots

Floor 2 → 30 slots

Floor 3 → 35 slots

Floor 4 → 40 slots

Java declaration:

```
int[] slotsPerFloor = {20, 25, 30, 35, 40};


Vehicle[][] slots = new Vehicle[5][];


for (int f = 0; f < slotsPerFloor.length; f++) {
   slots[f] = new Vehicle[slotsPerFloor[f]];
}
```

Each floor can have a different number of slots.

Therefore, a jagged array provides flexibility and avoids allocating unnecessary parking slots.

B. Evaluation of the Original Design

2 Marks

1. Can New Vehicle Types Be Added Using Subclasses?

Yes. Inheritance allows us to create new vehicle subclasses without duplicating all the common vehicle information.

For example:

- EBike extends Vehicle.
- DeliveryVan extends Vehicle.

However, delivery vans require time-based pricing, while e-bikes require flat pricing. A simple ratePerHour() method is not sufficient to represent every pricing rule cleanly.

Therefore, we introduce a separate PricingRule abstraction.

2. Are Pricing Rules Centralised?

In the original design, pricing rules are mainly implemented inside individual vehicle subclasses.

As the number of vehicle types increases, managing different pricing conditions becomes difficult.

The improved design introduces a PricingRule interface.

Different pricing classes implement this interface:

- HourlyPricingRule
- FlatFeePricingRule
- DeliveryVanPricingRule

This separates pricing calculations from vehicle information.

3. Can Existing Classes Remain Unchanged?

Some existing classes can remain unchanged, but certain classes require modifications.

The Vehicle class must be modified to support PricingRule.

The Car and TwoWheeler constructors must be updated to provide their pricing rules.

ParkingTicket must support different pass types.

ParkingFacility must support the updated five-floor configuration.

Therefore, the original design can be improved through limited modifications instead of rewriting the complete application.

C. Adaptation or Redesign

2 Marks

1. Final Design Decision

I choose to **MODIFY the original design**.

The original inheritance hierarchy is retained, but the pricing system is improved using a PricingRule interface.

A Pass interface is also introduced to support monthly and one-day parking passes.

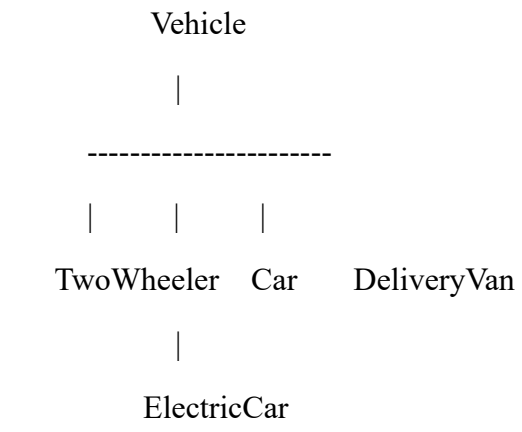
The revised design uses:

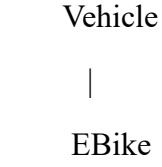
- Inheritance for vehicle categories.
- Composition for pricing rules.
- Interfaces for pricing and parking passes.

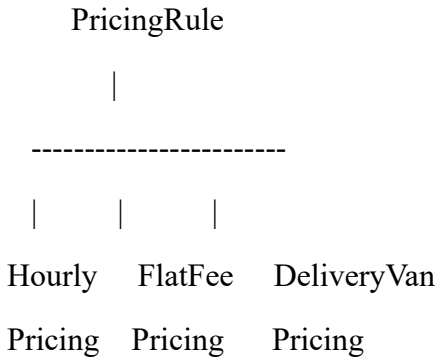
- Jagged arrays for parking floors.
- HashMap for active parking tickets.

2. Revised Class Structure

The updated class structure is:









MonthlyPass OneDayPass

Explanation of New Classes

1. EBike

Represents electric bikes and uses a flat parking fee.

2. DeliveryVan

Represents delivery vans and uses double the car rate between 9 AM and 5 PM.

3. PricingRule

An interface that defines the parking fee calculation.

4. FlatFeePricingRule

Calculates a fixed parking charge for e-bikes.

5. DeliveryVanPricingRule

Calculates delivery van charges according to the parking time.

6. Pass

An interface that checks parking pass validity.

7. MonthlyPass

Manages monthly passes for students and staff.

8. OneDayPass

Manages one-day passes purchased by visitors.

---

3. Pricing Assumptions

The original assignment does not specify the exact e-bike flat fee or car hourly rate.
Therefore, the implementation uses the following illustrative values:

- Two-wheeler: Rs. 10 per hour.
- Car: Rs. 30 per hour.
- Electric car: Rs. 30 per hour.
- E-bike: Rs. 20 flat fee per visit.
- Delivery van: Rs. 60 per hour from 9 AM to 5 PM.
- Delivery van: Rs. 30 per hour outside that period.
- EV charging: Rs. 12 per kWh.

Every started hour is charged fully.

For delivery vans, each started hourly parking block is charged according to the time at which that block begins. The 9 AM–5 PM period is treated as 9:00 inclusive and 17:00 exclusive.

A one-day pass is valid only for parking that begins and ends on its specified date.

6. Test Results for the New Requirements

The following tests demonstrate the newly introduced features.

Test 1 – E-bike

- Duration: 120 minutes.
- Flat parking fee: Rs. 20.
- Expected result: Rs. 20.

Test 2 – Delivery Van

- Entry: 9:00 AM.
- Exit: 10:30 AM.
- Billable hours: 2.
- Double car rate: Rs. 60 per hour.
- Expected result: Rs. 120.

Test 3 – Visitor One-Day Pass

- Entry: 11:00 AM.
- Exit: 1:00 PM.
- Pass validity: November 11, 2026.
- Expected parking fee: Rs. 0.

Test 4 – Five-Floor Facility

- Number of floors: 5.
- Slot capacities: 20, 25, 30, 35 and 40.
- Total capacity: 150 slots.
- Expected result: Different slot capacities are supported successfully.

Test 5 – Existing Vehicle Types

- Two-wheeler parking fee: Rs. 10.
- Electric car parking fee: Rs. 30.

- EV charging fee: Rs. 24.
- Expected result: Existing vehicle features continue to work.

After running the program, compare the actual output with the expected output and record your results.

D. Evidence-Based Defence

2 Marks

1. Number of Existing Classes That Had to Change

The redesigned implementation modifies five existing classes:

1. Vehicle – Updated to use the PricingRule interface.
2. TwoWheeler – Updated to provide its hourly pricing rule.
3. Car – Updated to provide its hourly pricing rule.
4. ParkingTicket – Updated to store the Pass interface.
5. ParkingFacility – Updated to support five floors and the revised pass system.

The previous ParkingPass class is replaced by the Pass interface with MonthlyPass and OneDayPass implementations.

New classes are introduced for E-bike, Delivery Van and their pricing rules.

The ElectricCar charging behaviour is retained.

2. Ease of Adding Future Vehicle Types

The new design makes future extensions easier.

For example, suppose the university introduces a bus category.

We can create a new Bus subclass and provide its required pricing rule.

The existing parking allocation and exit methods do not need to be rewritten.

Similarly, a new pricing strategy can be added by implementing the PricingRule interface.

3. Test Results

The redesigned system supports:

- E-bike flat parking fees.
- Delivery van time-based pricing.
- Visitor one-day passes.
- Student and staff monthly passes.

- Five floors with different slot capacities.
- Existing two-wheeler, car and electric car operations.

The example execution produces a total collection of Rs. 204.

This matches the manual calculation:

Rs. 20 + Rs. 120 + Rs. 0 + Rs. 54 + Rs. 10 = **Rs. 204**.

4. Trade-offs

Advantages:

1. New pricing rules can be added without changing the parking fee calculation logic.
2. Different vehicle categories are easier to manage.
3. Pass types can be extended through interfaces.
4. Jagged arrays support different floor capacities.
5. Composition improves flexibility and reduces dependence on vehicle subclasses for pricing.

Limitations:

1. Additional pricing classes increase the number of Java files or classes.
2. More objects and interfaces make the design slightly more complex.
3. Pricing assumptions must be clearly defined to avoid incorrect fee calculations.

5. Final Recommendation

I recommend modifying the original object-oriented design rather than completely replacing it.

The original inheritance hierarchy is retained, while pricing calculations are separated using the PricingRule interface. The Pass interface supports both monthly and one-day passes. A jagged array supports the extended five-floor parking facility.

The redesigned system is more flexible and maintainable because new vehicle categories and pricing rules can be introduced with minimal changes to existing parking operations.

---

7. Conclusion

The Smart-Parking System was successfully adapted to support the university's new requirements. E-bikes use flat-fee pricing, delivery vans use time-based pricing, visitors can purchase one-day passes, and the parking facility supports five floors with different slot capacities.

The modified design uses inheritance, abstraction, interfaces, composition and jagged arrays. Therefore, it provides a flexible foundation for future extensions without rewriting the complete system.